

TinyML Visual Alarm System: On-Device Person Detection with Notification

Andrea Renelli

Laboratory of Internet of Things, University of Trento

andrea.renelli@studenti.unitn.it

Abstract—Deploying ML inference directly on microcontrollers eliminates bandwidth overhead and privacy exposure inherent in cloud-based video surveillance. The main challenge is fitting a viable vision model within the severe memory and compute constraints of embedded hardware while maintaining alarm-grade precision. This paper presents a Smart Visual Alarm System on the ESP32-S3-EYE that addresses this challenge through fully on-device person detection: a custom MobileNetV1 retrained with quantization-aware training (QAT) running under TensorFlow Lite for Microcontrollers achieves 88.4% precision at a false-alarm rate of 8.9%, with end-to-end inference latency of 403 ms (≈ 2.48 FPS) on the embedded hardware. A three-frame moving-average filter further suppresses spurious triggers in live video. Alarm events are published via MQTT to a Python backend that delivers real-time Telegram notifications. Evaluated on 48,287 held-out COCO 2017 images, the system achieves ROC-AUC = 0.856 and AP = 0.877.

Index Terms—ESP32-S3-EYE, TFLite Micro, person detection, MQTT, Telegram Bot, IoT, edge AI, TinyML

I. INTRODUCTION

The proliferation of low-cost cameras in homes, offices, and public spaces has made video-based surveillance a cornerstone of modern security systems. Traditionally, these systems rely on continuous video streaming to remote servers or cloud platforms, where inference is performed centrally. This architecture, however, carries two fundamental costs. First, transmitting raw video over a network exposes sensitive visual data to interception, storage by third parties, and potential misuse, a serious privacy concern in residential and workplace deployments where occupants have a reasonable expectation not to be recorded and analysed off-premises, as exemplified by the well-documented Amazon Ring case [10]. Second, continuous video streaming is bandwidth-intensive: a single 720p camera at modest quality can consume several Mbps of uplink, making cloud-based inference economically and practically unviable in bandwidth-constrained or large-scale deployments. Edge computing addresses both issues by pushing inference directly to the data source, so that only high-level semantic events, rather than raw pixels, ever leave the device.

Realising vision-based inference on embedded hardware is, however, far from straightforward. Microcontrollers (MCUs) typically operate with tens to hundreds of kilobytes of SRAM, no dedicated GPU, and clock speeds in the hundreds of MHz, orders of magnitude below the resources assumed by standard deep learning frameworks. Fitting a convolutional neural network capable of meaningful person detection within these

constraints requires aggressive model compression (quantisation, architecture search) and careful memory management, all while meeting real-time latency requirements suitable for an alarm application. A complete Smart Visual Alarm System is presented that addresses the above challenges end-to-end on a single embedded device. The main contributions are: (i) a fully on-device person detection pipeline on the ESP32-S3-EYE, combining a custom QAT MobileNetV1 with a three-frame moving-average filter for robust alarm triggering without any cloud dependency, (ii) a lightweight MQTT-based event notification architecture in which raw video never leaves the device and only compact JSON descriptors are transmitted, (iii) a custom MobileNetV1 retrained with quantization-aware training that recovers the accuracy degraded by post-training quantisation, outperforming the stock PTQ int8 model (ROC-AUC=0.856 vs. 0.823 on the held-out test set) at zero additional inference cost, and (iv) a systematic evaluation on 48,287 held-out COCO 2017 images (training offset 70,000) demonstrating ROC-AUC = 0.856, AP = 0.877, and 88.4% precision at a false-alarm rate of 8.9% (firmware default $\theta = 0.60$), with a hardware inference latency of 403 ms on the target MCU.

II. SYSTEM ARCHITECTURE

A. Overview

The system comprises three tiers (Fig. 1): the ESP32-S3-EYE firmware, a Mosquitto MQTT broker, and a Python/Telegram backend. Raw video frames never leave the device: only compact JSON event descriptors are transmitted over the network. The firmware connects to the broker over Wi-Fi using MQTT QoS 1 for at-least-once alarm delivery. The notification pipeline follows a *Publisher-Subscriber* pattern, decoupling event generation from consumers.

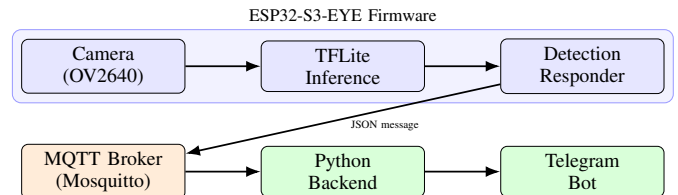


Fig. 1. System architecture. Raw video stays on-device: only JSON alarm events are published over MQTT.

B. Hardware Platform: ESP32-S3-EYE

The target hardware is the ESP32-S3-EYE development board by Espressif Systems [1]. The main specifications are summarised in Table I.

TABLE I
ESP32-S3-EYE HARDWARE SPECIFICATIONS.

Component	Specification
SoC	ESP32-S3, dual-core Xtensa LX7 @ 240 MHz
On-chip SRAM	512 KB
External PSRAM	8 MB (OPI PSRAM)
Flash	8 MB
Camera	OV2640, up to 2 MP
Connectivity	Wi-Fi 802.11 b/g/n, Bluetooth 5.0
Display	1.3-inch LCD (240×240)
Microphone	Digital PDM microphone
Power supply	USB, 5 V

C. On-Device Inference

TFLite Micro [2] operates on a statically allocated 100 KB tensor arena placed in PSRAM to preserve on-chip SRAM for the RTOS and network stacks (relocating the arena to internal SRAM yielded no measurable latency improvement and disrupted LCD DMA transfers, so PSRAM allocation was retained). Input images are resized to 96×96 pixels and converted to signed 8-bit integers before feeding the quantized MobileNetV1 model. The model outputs a *person score* dequantised to float. The deployed model is a custom MobileNetV1 retrained with quantization-aware training (QAT), replacing the stock post-training-quantised model shipped with the esp-tflite-micro repository [3].

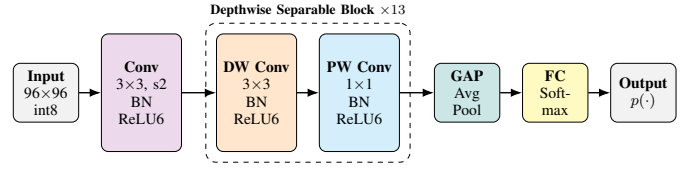
1) *MobileNetV1 Architecture*: MobileNetV1 [5] is a lightweight convolutional neural network designed for deployment on resource-constrained devices. Rather than relying on standard convolutions throughout, it introduces *depthwise separable convolutions* as the main building block. A depthwise separable convolution factorises a standard $k \times k$ convolution into two sequential operations: a per-channel 3×3 *depthwise* filter, which applies a single filter to each input channel independently, followed by a 1×1 *pointwise* convolution that linearly combines the outputs across channels. This factorisation yields a reduction in multiply-accumulate operations (MACs) with respect to a standard convolution of:

$$\frac{\text{DW-Sep MACs}}{\text{Std Conv MACs}} = \frac{1}{C_o} + \frac{1}{k^2} \approx \frac{1}{9} \quad (k=3, C_o \gg 1) \quad (1)$$

making MCU deployment feasible without a substantial loss in representational capacity.

The architecture (Fig. 2) opens with one 3×3 convolution followed by 13 depthwise separable blocks, global average pooling, and a softmax classifier. The depth multiplier $\alpha=0.25$ scales all channel counts uniformly, yielding 213,272 parameters and ≈ 7.5 MMAC.

The original esp-tflite-micro model uses *post-training linear quantization* (PTQ) [2]: each weight is mapped to a signed 8-bit integer via $r = S(q - Z)$, where S is a per-channel scale and Z is the zero-point. This compresses storage



Depth multiplier α scales channels uniformly (deployed: $\alpha=0.25$).
Spatial downsampling via strided DW conv – no max pooling.

Fig. 2. MobileNetV1 architecture. One standard convolution opens the network. Thirteen depthwise separable blocks follow, each consisting of a 3×3 depthwise conv, a 1×1 pointwise conv, batch normalisation, and ReLU6 activation. Global average pooling and a fully connected softmax classifier close the network.

without additional training, but calibrating quantisation scales post-hoc introduces rounding noise that the model was not trained to tolerate, degrading accuracy (see Section III-D).

2) *Quantization-Aware Training*: QAT [8] inserts *fake-quant* operators during training so the model learns weight distributions robust to 8-bit rounding, closing the accuracy gap introduced by PTQ.

a) *Training pipeline*.: Training follows a two-phase transfer-learning strategy on 70,000 COCO 2017 images. *Phase 1* (5 epochs, Adam, $\text{lr} = 10^{-3}$): the ImageNet-pretrained backbone is frozen and only the new classifier head is trained. *Phase 2* (5 epochs, $\text{lr} = 10^{-4}$): the full network is unfrozen and fine-tuned end-to-end. EarlyStopping (patience = 4) and ReduceLROnPlateau (factor = 0.5, patience = 2) are applied throughout, with ModelCheckpoint saving the best validation-accuracy weights. QAT then fine-tunes the float32 checkpoint for 3 additional epochs at $\text{lr} = 10^{-5}$, a low rate that preserves the learned weights while adapting them to quantisation noise.

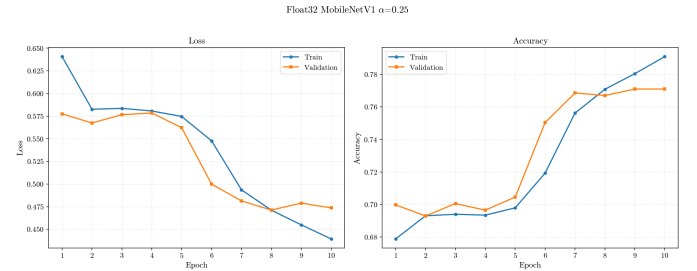


Fig. 3. Float32 MobileNetV1 ($\alpha=0.25$) training curves (10 epochs, 70,000 COCO 2017 images). The visible kink at epoch 5 marks the transition from Phase 1 (backbone frozen, head only) to Phase 2 (full fine-tuning at $\text{lr} = 10^{-4}$).

Three adaptations bridge the grayscale sensor and the pre-trained backbone: (i) a Concatenate layer replicates the single grey-scale channel three times (CONCATENATION op in TFLite), this is needed to adapt the model trained with ImageNet (RGB input), (ii) GlobalAveragePooling2D exports as MEAN and (iii) a Dense (2) softmax head replaces the stock 1×1 Conv2D classifier (FULLY_CONNECTED + SOFTMAX).

b) *Float32 / PTQ / QAT accuracy.*: Table II reports metrics on the COCO 2017 validation set, confirming that the fake-quant regularisation is beneficial rather than merely neutral.

TABLE II

MODEL VARIANTS EVALUATED ON 5,000 HELD-OUT IMAGES. ACCURACY AT $\theta = 0.5$; LOSS FROM `MODEL.EVALUATE()`

Model	AUC-ROC	Δ AUC	Acc.	Loss
Float32	0.859	—	77.3 %	0.467
PTQ int8 (stock)	0.855	−0.004	76.1 %	—
QAT int8 (ours)	0.862	+0.003	77.4 %	0.468

The resulting graph adds ≈ 13 ms of overhead over the PTQ backbone, yielding a total invoke time of 403 ms at a model size of 304.2 KB int8 (+10.7 KB vs. PTQ).

D. Detection Logic and Alarm Trigger

A moving-average filter with window $W = 3$ frames smooths transient noise:

$$\bar{s}_t = \frac{1}{W} \sum_{k=0}^{W-1} s_{t-k} \quad (2)$$

At ≈ 2.48 FPS, $W = 3$ spans ≈ 1.2 s, sufficient to suppress single-frame false positives while preserving sustained detections. An alarm fires when $\bar{s}_t \geq \theta$ and at least $C_{\min}$ seconds have elapsed (defaults: $\theta = 0.60$, $C_{\min} = 10$ s); all three parameters are exposed as `menuconfig` options. On alarm, the firmware blinks the onboard LED and publishes:

```
{"event_id":42, "confidence":0.87, "timestamp_ms":123456789}
```

Fig. 4 illustrates the smoothing effect of the moving-average filter on a representative 120-frame sequence captured during a live deployment.

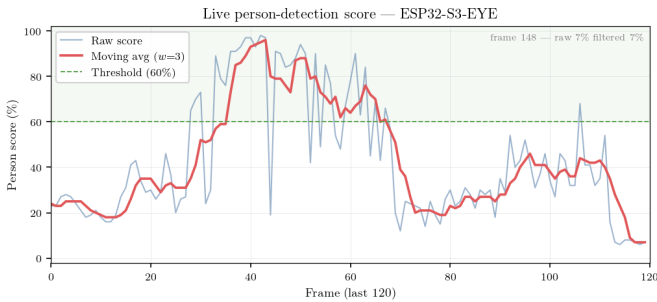


Fig. 4. Raw per-frame person score (blue) and its 3-frame moving average (red) recorded on the ESP32-S3-EYE during a live session. The green dashed line marks the alarm threshold $\theta = 0.60$. The filter attenuates transient spikes.

E. Python Backend and Telegram Notification

A Python script subscribes to the MQTT topic `paho-mqtt`, validates each alarm payload, plays a local audio alert, and dispatches a notification via the Telegram Bot API containing the event ID, confidence, and timestamp. Credentials are passed via environment variables.

III. EXPERIMENTAL RESULTS

A. Hardware Inference Latency

Inference timing was collected on the deployed device (Xtensa LX7 @ 240 MHz, `CONFIG_NN_OPTIMIZED` enabled) over $n = 104$ consecutive frames. Results are shown in Table III and Fig. 5.

TABLE III

INFERENCE LATENCY ON ESP32-S3-EYE ($n = 104$ FRAMES).

Layer group	Mean	Std	Min	Max
Conv2D (3×3 + all 1×1 PW)	367.8 ms	1.04 ms	366 ms	370 ms
DepthwiseConv2D	22.3 ms	0.38 ms	22 ms	24 ms
Other (GAP, QUANTIZE, FC, Softmax)	13.4 ms	0.21 ms	13 ms	14 ms
Total Invoke()	403.5 ms	1.12 ms	401 ms	406 ms

Conv2D (opening layer + 13 pointwise 1×1) accounts for $\approx 91\%$ of invoke time. DepthwiseConv2D contributes $\approx 6\%$, while QAT-specific and classifier ops account for the remaining $\approx 3\%$ (Fig. 5). Early DS blocks on larger feature maps are up to $3 \times$ slower than later ones (DS 1–3 vs DS 6–13). Total: 403 ms (≈ 2.48 FPS).

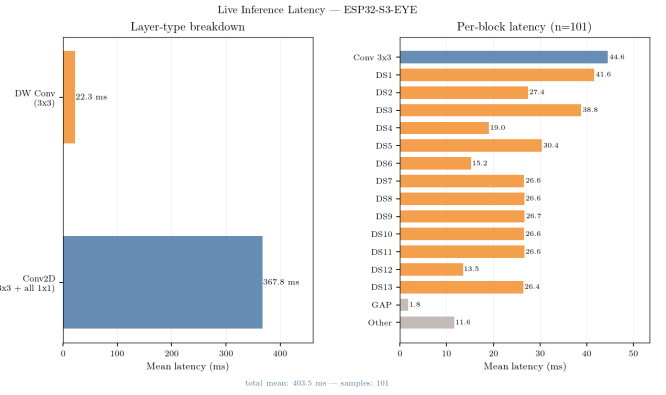


Fig. 5. Inference latency breakdown on the ESP32-S3-EYE QAT model ($n = 104$ frames). *Left*: Conv2D (3×3 opening + all 1×1 PW, 367.8 ms, $\approx 91\%$) vs. DW Conv (22.3 ms, $\approx 6\%$). *Right*: per-block mean latency. Early DS blocks (DS 1–3) on larger feature maps are up to $3 \times$ slower than later ones (DS 6–13).

B. Model Efficiency Metrics

TinyML efficiency metrics for the deployed MobileNetV1 model are reported in Table IV alongside MobileNetV2 for direct comparison. #Parameters counts all weight and bias tensors reported by the TFLite interpreter. Model size is taken directly from the C-array length in `person_detect_model_data.cc`. Estimated MACs follow analytically from Eq. (1) applied to each layer of the MobileNetV1 ($\alpha=0.25$) graph.

As noted in [6], activation memory (tensor arena) is the binding constraint on MCUs, not parameter count.

C. MobileNetV2 Candidate: Hardware Latency

As a candidate for higher accuracy, a MobileNetV2 model trained for person detection on the ESP32-S3 [9] was ported to

the firmware and measured on the same hardware over $n=96$ consecutive frames. Table IV compares the two architectures across all TinyML efficiency metrics.

TABLE IV
TINYML EFFICIENCY METRICS: MOBILENETV1 VS. MOBILENETV2. V2 LATENCY MEASURED ON-DEVICE ($n=96$ FRAMES, LATENCY_LOG.CSV).

Metric	MobileNetV1	MobileNetV2
Input resolution	96×96, gray, int8	160×160, RGB, int8
#Parameters	210,856	453,104
Model size (int8)	304.2 KB	664.5 KB
Float32 equivalent	1,217 KB	2,658 KB
Estimated MACs	≈7.5 MMAC	≈26.7 MMAC
Estimated FLOPs	≈15.0 MFlop	≈53.3 MFlop
Tensor arena (PSRAM)	100 KB	900 KB
Inference latency (mean)	403.5 ms	1,919.4 ms
Inference latency (std)	1.12 ms	0.71 ms
Inference latency (min)	401 ms	1,919 ms
Inference latency (max)	406 ms	1,921 ms
Throughput	2.48 FPS	0.52 FPS

MobileNetV2 [7] uses inverted residual bottlenecks with an expansion factor of 6, raising MACs by $3.7\times$ and the tensor arena from 100 KB to 900 KB. Combined with the $8.3\times$ larger input, invoke time reaches 1,919 ms (0.52 FPS), a $4.8\times$ slowdown. MobileNetV2 was therefore not evaluated for detection quality on this hardware.

D. Detection Quality

Both models (v1 in both PTQ and QAT versions) were evaluated on the held-out test set: the 48,287 COCO 2017 train images from index 70,000 onward, i.e. the portion never seen during QAT training. Labels follow the Visual Wake Words (person / no-person) layout [4]. Preprocessing replicates the firmware pipeline: BT.601 luma conversion and signed-8-bit quantisation.

1) *PTQ vs. QAT accuracy comparison*: Table V compares the stock PTQ model against the custom QAT model on the held-out test set: COCO 2017 train images from index 70,000 onward (48,287 images), never seen during QAT training. At $\theta=0.60$, QAT reduces FAR from 11.6 % to 8.9 % (−2.7 pp) and raises precision from 85.6 % to 88.4 % (+2.8 pp) with a negligible F1 change (+0.002). AUC rises from 0.823 to 0.856 (+0.033).

TABLE V
PTQ VS. QAT ON 48,287 HELD-OUT COCO 2017 IMAGES ($\theta=0.60$, IMAGES[70000:]).

Model	AUC	AP	Prec.	Recall	F1	FAR
PTQ int8 (stock)	0.823	0.858	85.6 %	58.4 %	0.695	11.6 %
QAT int8 (ours)	0.856	0.877	88.4 %	57.5 %	0.697	8.9 %
Δ (QAT−PTQ)	+0.033	+0.019	+2.8 pp	−0.9 pp	+0.002	−2.7 pp

2) *Test-set evaluation*: Overall, ROC-AUC=0.856 and AP=0.877 on the held-out test set (Figs. 7, 8, 9, 10) confirm good discriminative capacity despite int8 quantisation. The

firmware-default $\theta=0.60$ was selected by combining benchmark results with on-site validation in a typical indoor environment. At $\theta=0.60$, precision is 88.4 % with FAR=8.9 %. Per-frame recall (57.5 %) is partially compensated by the moving-average filter: on sustained person presence consecutive frames produce correlated high scores that consistently exceed the threshold, while isolated false positives are averaged down by surrounding low-score frames, effectively reducing the live false-alarm rate below the per-frame FAR reported here.

Fig. 6 shows the confusion matrices: at $\theta=0.60$ false negatives far outnumber false positives, confirming the system errs toward silence. COCO 2017 covers a wide range of non-surveillance scenes and predominantly 4:3 images whose square-crop resizing distorts person silhouettes [3], so per-frame metrics underestimate operational performance.

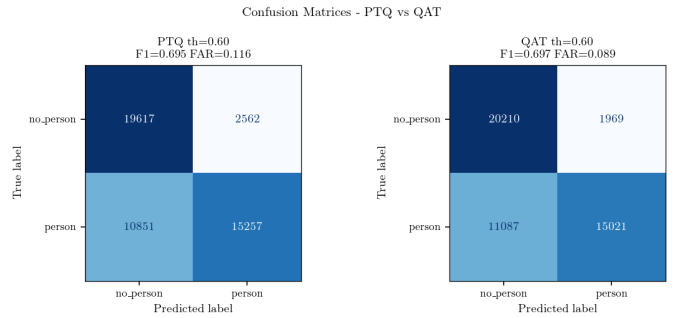


Fig. 6. Confusion matrices at the firmware default $\theta=0.60$: PTQ int8 (left, FAR=11.6 %) vs. QAT int8 (right, FAR=8.9 %). QAT reduces false positives by 593 on the 48,287-image test set.

IV. CONCLUSION

This paper presented a complete Smart Visual Alarm System running fully on the ESP32-S3-EYE: on-device TFLite Micro inference, moving-average alarm logic, MQTT QoS 1 event publication, and Python/Telegram notification. Raw video is never transmitted: only compact JSON descriptors leave the device. A custom MobileNetV1 was retrained with quantization-aware training, recovering the accuracy degraded by post-training quantisation: on the held-out test set (48,287 COCO 2017 images, offset 70,000), QAT int8 achieves AUC=0.856 and AP=0.877 vs. AUC=0.823 and AP=0.858 for the stock PTQ int8, with a measured inference latency of 403 ms (≈2.48 FPS) and a 304 KB model footprint (Table V). The firmware threshold $\theta=0.60$ provides 88.4 % precision at 8.9 % FAR, and the three-frame moving-average filter further reduces the effective false-alarm rate on live video.

Future work includes replacing the backbone with MCUNet [6], which targets higher accuracy at comparable or lower memory and MAC budgets, and re-benchmarking on a surveillance-specific dataset covering typical indoor and outdoor camera viewpoints to obtain accuracy estimates more representative of operational deployments.

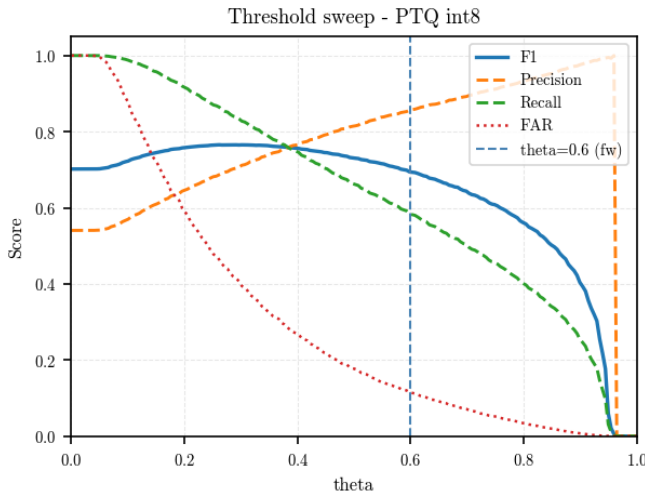


Fig. 7. F1, precision, recall, and FAR vs. θ for the PTQ int8 model. Dashed: firmware default ($\theta=0.60$).

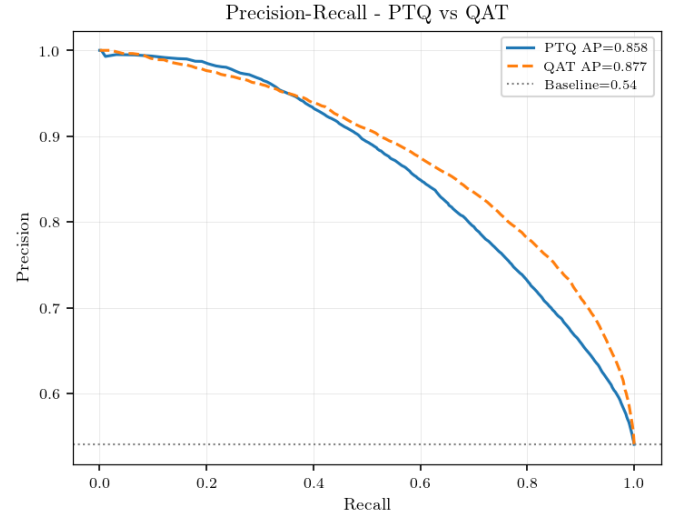


Fig. 9. Precision-Recall curves for PTQ (AP=0.858) and QAT (AP=0.877).

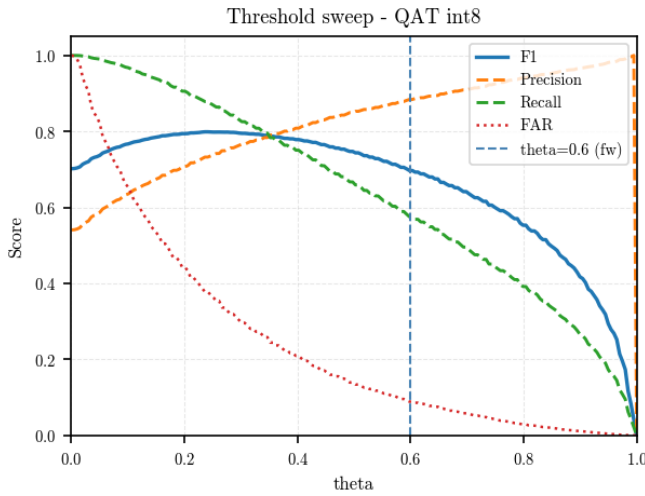


Fig. 8. F1, precision, recall, and FAR vs. θ for the QAT int8 model. Compared to PTQ, precision is higher and FAR lower at every threshold. Dashed: firmware default ($\theta=0.60$).

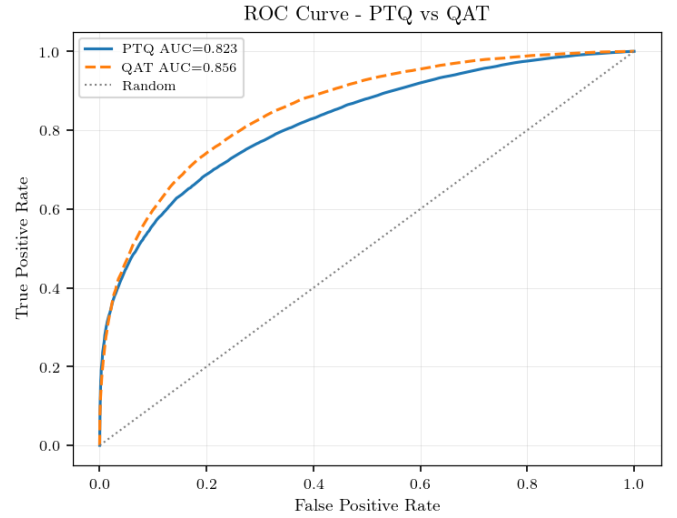


Fig. 10. ROC curves for PTQ (AUC=0.823) and QAT (AUC=0.856). Dashed diagonal: random classifier.

REFERENCES

- [1] Espressif Systems, “ESP32-S3-EYE Getting Started Guide,” 2022. [Online]. Available: <https://github.com/espressif/esp-who>
- [2] R. David *et al.*, “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems,” in *Proc. MLSys*, 2021.
- [3] Espressif Systems, “esp-tflite-micro,” GitHub, 2023. [Online]. Available: <https://github.com/espressif/esp-tflite-micro>
- [4] A. Chowdhery *et al.*, “Visual Wake Words Dataset,” *arXiv:1906.05721*, 2019.
- [5] A. Howard *et al.*, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [6] J. Lin *et al.*, “MCUNet: Tiny Deep Learning on IoT Devices,” in *NeurIPS*, 2020.
- [7] M. Sandler *et al.*, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. CVPR*, 2018.
- [8] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proc. CVPR*, 2018.
- [9] ai4iot, “Person Detection on ESP32-S3,” GitHub, 2023. [Online]. Available: <https://github.com/ai4iot/person-detection-esp32s3>
- [10] Federal Trade Commission, “FTC Says Ring Employees Illegally Surveilled Customers, Failed to Stop Hackers from Taking Control of Users’ Cameras,” Press Release, May 2023. [Online]. Available: <https://www.ftc.gov/news-events/news/press-releases/2023/05/ftc-says-ring-employees-illegally-surveilled-customers-failed-stop-hackers-taking-control-users>